



Milestone 2

Individual Evidence Pack

- **College:** College of Computing
- **Major:** Computer Science
- **Group Number:** Project Team 2
- **Student Name:** CHEN Xian
- **Student Number:** 59790558
- **Milestone:** M2

Polymarket signal analysis — detect unusual odds movements empirically

1 What I Contributed

- Extended the data collection pipeline from snapshot-only collection to dual-mode collection (snapshot + trade-level records), including orchestration logic for both YES and NO outcomes in the main runtime loop.
- Implemented and refined incremental trade collection with catch-up behavior and pagination safety limits, so the collector can resume from prior state and avoid infinite loops.
- Added/updated trade storage components and schema usage (trade files and cursor persistence), then integrated these outputs into the project dataset for downstream analysis.
- Tuned collection/runtime settings for reliability under larger requests (timeouts, retries, and market-scope behavior), and aligned configuration with optional L2 credential flow.

2 Evidence

- Code implementation evidence (core pipeline and logic):
 - ``getdata/main.py``: orchestrates market-level processing and integrates snapshot + trade collection flow.
 - ``getdata/core/trade_collector.py``: handles incremental trade fetching, cursor continuation, and catch-up pagination logic.
 - ``getdata/core/storage.py``: handles persistent storage for trade records and related outputs.
 - ``getdata/config/settings.py``: runtime and collection parameters (timeouts, retries, collection scope, optional credentials).
- Data artifact evidence (generated outputs in repository):
 - ``polymarket_data/market_metadata.csv``
 - ``polymarket_data/timeseries/*.csv``
 - ``polymarket_data/trades/*.csv``
 - ``polymarket_data/trade_cursors.json``
 - ``polymarket_data/logs/collection.log``
- GitHub Repository: <https://github.com/SHANECHEN0722/pet-getdata.git>

3 Validation Performed

Validation item: CSV schema and integrity check against current data artifacts.

What was tested:

Trade schema completeness (11 required fields): timestamp, datetime, trade_id, market_id, token_id, side, price, size, amount, tx_hash, raw.

Snapshot and metadata structure presence.

Sample trade ID uniqueness (duplicate check on one YES trade file).

Method:

Ran a Python validation script on the repository data files (reading market metadata, one timeseries sample, and YES/NO trade samples).

Result:

- trade_header_ok = True
- metadata_cols = 11, metadata_rows = 6
- timeseries_cols = 8, timeseries_rows = 3343
- trade_yes_cols = 11, trade_yes_rows = 282
- trade_no_cols = 11, trade_no_rows = 3421
- trade_yes_unique_ids = True
- Conclusion: PASS. Sampled artifacts are structurally complete and consistent with the intended schema.